

Aurum Market

Descubrimiento semántico y control de duplicados sobre un catálogo de 15.000 productos

Módulo 10 · Bases de Datos Vectoriales · Pontia · Qdrant 1.18 · multilingual-e5-base · 768 dimensiones · coseno · Sin LLM en tiempo de ejecución

1 · El problema

Aurum Market vende de todo y su buscador solo encuentra lo que se escribe con las palabras exactas de la ficha. Dos recorridos concretos fallan: un cliente que describe lo que necesita en lugar de nombrarlo se va sin comprar, y el equipo de catálogo publica duplicados que nadie detecta hasta que un cliente se queja.

Ambos son el mismo problema técnico —comparar significados en vez de cadenas— y por eso se resuelven con una sola base vectorial. La restricción de partida es que **no puede intervenir ningún LLM en tiempo de ejecución**: lo que se evalúa es si la representación y el índice, por sí solos, sostienen las decisiones.

El riesgo real de un encargo así no es que no funcione, sino que funcione *sin que se pueda demostrar por qué*. El desarrollo se organizó alrededor de esa idea: cada decisión se contrasta con una medición, y cada medición viene con la evidencia que permite reproducirla.

2 · Cómo se mide

Los juicios de relevancia son **graduados**, no binarios: cada par consulta-producto está etiquetado como Exact (3), Substitute (2), Complement (1) o Irrelevant (0). Se cuentan como relevantes los grados ≥ 2 —lo que el cliente buscaba y lo que le sirve igual, pero no un accesorio compatible—, y ese umbral se declaró antes de medir nada.

Conjunto	Elementos	Para qué
Catálogo	15.000	Los productos indexados
Consultas de desarrollo	8	Ajustar y decidir. Con relevancias visibles
Juicios de relevancia	248	La verdad de referencia sobre desarrollo
Consultas ciegas	12	Se entregan sin conocer su respuesta
Altas de desarrollo / ciegas	14 / 14	Calibrar y predecir duplicados
Eventos de catálogo	24	Prueba operativa de mutación

HALLAZGO · EL TECHO ESTRUCTURAL

Recall@10 **no puede superar 0,5188** con estos datos. Las ocho consultas tienen una media de ~25 productos relevantes y solo hay diez posiciones disponibles. Es un límite aritmético, no una carencia del sistema. Sin declararlo, un Recall de 0,25 se lee como un fracaso; con él, es el 48 % de todo lo alcanzable.

Todo se promedia **por consulta y luego entre consultas** (macro-promedio). Promediar sobre el total de pares dejaría que las consultas con muchos juicios decidieran el resultado por sí solas.

3 · Alternativas probadas

3.1 · Representación: cuatro configuraciones, una variable cada vez

Cada experimento cambia **una sola cosa** respecto al anterior, y todos corren contra el oráculo exacto —sin índice aproximado— para que ninguna diferencia pueda atribuirse al motor.

Exp	Texto codificado	Modelo	nDCG@10	Recall@10	MRR@10
E0	text completo	TF-IDF (baseline)	0,6198	0,3031	0,9167
E1	text completo	e5-small	0,6427	0,2830	0,8125
E2	título + marca + color	e5-small	0,6881	0,3277	0,8542
E3	título + marca + color	e5-base	0,7072	0,3648	0,9375

HALLAZGO · EL TEXTO PESÓ MÁS QUE EL MODELO

Arreglar qué se codifica (E1→E2) aportó **+0,045** de nDCG. Duplicar el tamaño del modelo (E2→E3) aportó **+0,019**: menos de la mitad, a cambio de duplicar la dimensión y el coste.

Y algo más incómodo: **E1 pierde contra el baseline léxico en Recall y en MRR**. Adoptar embeddings sin tocar el texto habría *empeorado* el sistema. El campo `text` del catálogo mide 1.309 caracteres de media y el 27 % excede la ventana de 512 tokens del modelo: se estaba codificando ruido, y truncándolo.

3.2 · Índice: barrido de `ef_search`

Los parámetros `m` y `ef_construct` gobiernan el grafo y cambiarlos obliga a reconstruirlo. `ef_search` es el único ajustable por consulta, así que es el que se barre. Fidelidad = solapamiento de IDs con el oráculo exacto sobre el catálogo completo.

<code>ef_search</code>	Fidelidad@10	Orden idéntico	Operación	p50
16	0,8500	0,7000	Qdrant, <code>ef_search=16</code>	44,00 ms
32	0,8875	0,7250	Qdrant, <code>ef_search=256</code>	47,16 ms
64	0,9750	0,8750	Qdrant, solo contar puntos	32,64 ms
128	0,9875	0,9250	Oráculo NumPy, fuerza bruta	18,43 ms
256	1,0000	1,0000	Codificar la consulta	2,56 ms

Se elige **256**: cuesta 3,16 ms más que 16 y compra fidelidad y orden perfectos. Esa tercera fila resaltada es la que explica la elección — una operación que *no busca nada* tarda 32,64 ms, así que el transporte HTTP domina la medición y la búsqueda real son unos 8 ms. Optimizar `ef_search` a la baja no habría comprado latencia, solo perdido precisión.

3.3 · Duplicados: dónde poner la frontera

La regla se calibró sobre las 14 altas de desarrollo barriando el umbral. Las dos clases **no se solapan**:

```
duplicados reales    [0,9484 ... 0,9662]
                        hueco de 0,0586
productos nuevos     [0,8521 ... 0,8898]

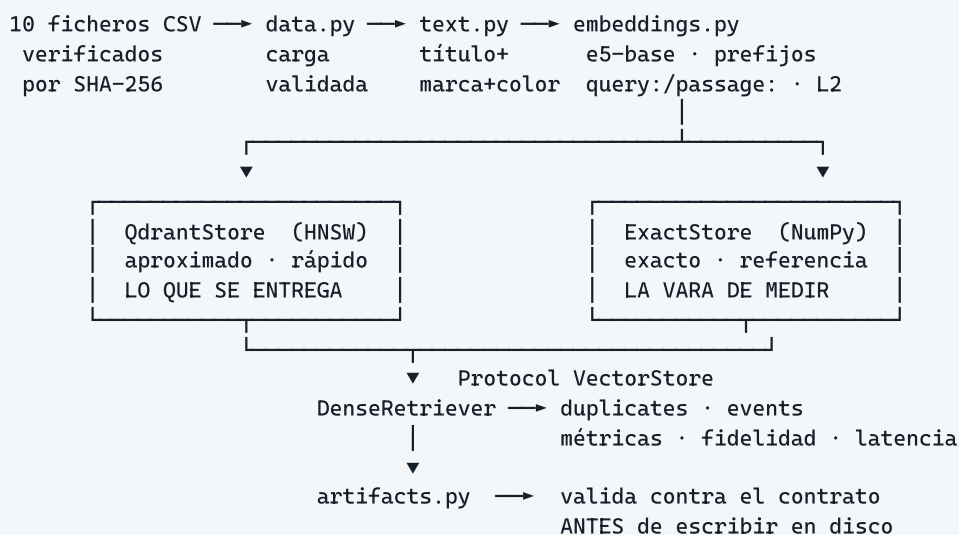
umbral elegido = 0,9191  (punto medio: el margen más ancho a ambos lados)
```

Se probó añadir el margen respecto al segundo candidato a la regla y **no discrimina**: es igual de estrecho en ambas clases. Se descarta y se documenta, porque una regla más compleja sin ganancia medible es deuda.

HALLAZGO · LO DECISIVO NO FUE LA REGLA, SINO EL FORMATO

Comparar el `text` en bruto de la ficha entrante contra un catálogo indexado como título+marca+color compara dos cosas distintas. Al componer la ficha entrante *igual que el catálogo*, la separación entre clases pasó de 0,0032 a 0,0586: dieciocho veces más ancha, con la misma regla y el mismo modelo.

4 · Arquitectura final



La decisión que sostiene todo lo demás es que existan **dos almacenes sobre exactamente los mismos vectores**. Implementan el mismo `Protocol`, así que el código de evaluación no puede distinguirlos: mismo modelo, mismo texto, mismos vectores, misma métrica. Difieren solo en cómo encuentran los vecinos.

Esa igualdad forzada convierte cualquier diferencia entre ellos en un diagnóstico:

```
el oráculo acierta y el motor no → la pérdida es del ÍNDICE
los dos fallan igual              → el problema es de REPRESENTACIÓN
```

La colección

```
aurum-market-catalogo
├─ vectores    768 dim · COSINE · normalizados
├─ ID          record_id = uuid5(namespace, product_id)
│              └─ estable ⇒ reingresar SOBRESERIBE, nunca duplica
├─ payload     product_id · title · brand · color · locale
│              catalog_version · active      (nulos = "", nunca "nan")
└─ índices     HNSW m=24 ef_construct=120 ef_search=256
                KEYWORD(brand) ⇒ el filtro lo resuelve el motor
                umbrales de indexación = 1.000 KB POR SEGMENTO
```

HALLAZGO · EL MOTOR RESPONDÍA SIN USAR SU ÍNDICE

Tras la primera ingesta, la colección estaba `green`, respondía correctamente... y tenía `indexed_vectors_count = 0`. Los umbrales de Qdrant se expresan en **kilobytes por segmento**, no en número de vectores, y con el valor por defecto el grafo nunca llegaba a construirse.

De no haberlo detectado, `m` y `ef_construct` habrían sido decoración, la fidelidad ANN habría salido trivialmente 1,0 y la latencia habría medido un recorrido lineal. Por eso la verificación **lee la configuración de vuelta del motor** en lugar de confiar en lo que se le pidió.

5 · Resultados

Recuperación · desarrollo	Valor	Operaciones	Valor
nDCG@10	0,5422	Consultas filtradas conformes	4 / 4
Recall@10	0,2481	...productos de la marca pedida	10/10
...sobre un techo de	0,5188	Duplicados · P / R / F1 (desarrollo)	1,0 / 1,0 / 1,0
MRR@10	0,9167	Eventos aplicados	24
Fidelidad ANN@10	1,0000	Reaplicación · estado	idéntico
Latencia p50 / p95	59 / 85 ms	Pruebas automáticas	379

MRR de 0,9167 con nDCG de 0,5422 describe un sistema con un perfil muy concreto: casi siempre acierta en la primera posición, y luego se queda corto llenando el resto del top-10. Para un buscador de comercio electrónico esa es la mitad buena del reparto, pero limita el descubrimiento.

La **fidelidad de 1,0** tiene una consecuencia fuerte y poco cómoda: el índice no pierde ni un solo candidato frente al oráculo exacto, así que *ningún* fallo de ranking puede achacársele. Todo el error observado es de la representación.

Latencia, desglosada

Componente	p50	p95	Lectura
Codificar la consulta	0,50 ms	0,97 ms	despreciable con el modelo ya cargado
Recorrido completo	81,77 ms	102,21 ms	codificación + red + búsqueda

2 repeticiones de calentamiento, 5 de medida, 8 consultas · Windows 11, 8 núcleos, Qdrant en Docker Desktop sobre WSL2 · `ef_search` = 256.

HALLAZGO · A ESTA ESCALA EL ÍNDICE ANN NO COMPENSA

El oráculo por fuerza bruta en NumPy resuelve la misma consulta en **18,43 ms** frente a los 47,16 ms de Qdrant, y con precisión exacta. Con 15.000 vectores de 768 dimensiones —23 MB que caben holgadamente en memoria— la estructura de índice no tiene nada que ganar.

Esto no invalida la elección: se pedía explícitamente una base de datos vectorial, y lo que Qdrant aporta —persistencia, filtrado nativo por payload, mutaciones concurrentes— no lo da un array de NumPy. Pero la ventaja *no* es la velocidad de búsqueda, y decirlo al revés sería falso.

6 · Análisis de fallos

Cinco fallos representativos, atribuidos a capa con su evidencia. Que la fidelidad sea 1,0 deja al índice sin responsabilidad en los fallos reales, así que el fallo de índice hubo que **provocarlo** —bajando `ef_search` a 16— para demostrar que el diagnóstico sabe distinguir esa capa.

Caso	Capa	Evidencia que lo sostiene
DEV-33633	representación	nDCG 0,238. El oráculo exacto recupera los <i>mismos</i> 2 relevantes que el motor
DEV-31224	representación	nDCG 0,362. Motor y oráculo coinciden en 3 de 15 relevantes
DEV-38249	representación	nDCG 0,379 con 35 relevantes juzgados. Idéntico en ambos
DEV-28703	índice (provocado)	Con ef_search=16 se pierden 3 productos que el oráculo sí encuentra
EVAL-DUP-004	datos / umbral	score 0,8907 < umbral 0,9191, pero <i>dentro</i> del hueco de desarrollo

HALLAZGO · EL SISTEMA SE DEGRADA JUSTO DONDE DEBERÍA BRILLAR

"taladro 24v batería" devuelve taladros. Su paráfrasis "quiero una herramienta inalámbrica potente para perforar sin depender de un enchufe" devuelve *guantes y estanterías de cocina*.

El diagnóstico se hizo por eliminación: existen 70 taladros en el catálogo, los prefijos query: / passage: se aplican correctamente, y el **oráculo exacto falla igual**. No es el índice ni la configuración: el modelo no acerca esa paráfrasis a las fichas. Codificar solo título+marca+color mejoró las métricas globales y, a la vez, eliminó el contexto que una consulta descriptiva necesita.

No se corrigió deliberadamente. Ajustar la representación contra las consultas de evaluación sería optimizar sobre el examen.

Duplicados sobre el conjunto ciego

Con el umbral congelado antes de ver un solo caso: **13 aciertos de 14**. Los 7 productos nuevos se clasificaron correctamente y 6 de los 7 duplicados se detectaron.

El fallo, `EVAL-DUP-004`, puntuó **0,8907** — por debajo del umbral, pero *dentro* del hueco [0,8898 – 0,9484] que en desarrollo estaba vacío. No es un error de la regla: es la prueba de que **catorce casos son demasiado pocos** para fijar una frontera, por limpia que parezca la separación. El umbral no se retocó al descubrirlo, porque hacerlo habría invalidado la evaluación.

7 · Recomendación

El sistema es apto para producción en el recorrido de **duplicados** y apto con reservas en el de **búsqueda**.

Recorrido	Veredicto	Condición
Control de altas duplicadas	Desplegable	Revisión humana en la banda 0,88–0,95, donde vive el único fallo observado
Búsqueda por palabras cercanas al catálogo	Desplegable	MRR 0,92: el primer resultado acierta
Búsqueda por descripción libre	No desplegable	Falla de forma visible y vergonzante ante paráfrasis

Qué haría a continuación, por orden de retorno

- **Enriquecer el texto codificado sin devolver el ruido.** El salto de `text` a título+marca+color arregló las métricas y rompió las consultas descriptivas. Lo siguiente es una composición intermedia —título + marca + color + las primeras características, con el keyword-stuffing filtrado— evaluada con el mismo protocolo. Es donde el proyecto ya demostró que está la mayor ganancia por euro invertido.
- **Recuperación híbrida.** TF-IDF gana al sistema denso en MRR sobre estas consultas. Fusionar ambos rankings recupera lo que cada uno ve y el otro no, sin cambiar de modelo ni reingerir.
- **Ampliar el conjunto de calibración de duplicados.** Catorce casos no bastan; con cincuenta, el umbral dejaría de ser una interpolación entre dos nubes pequeñas.
- **Medir con consultas reales de los clientes.** Las ocho de desarrollo son cortas y léxicas, lo que favorece al baseline y esconde exactamente el fallo que el sistema tiene.

Qué cambiaría con un catálogo diez veces mayor

Dimensión	Hoy · 15.000	150.000
Memoria de vectores	23 MB	~230 MB · sigue cabiendo en RAM
Codificación inicial	~12 min en CPU	~2 h en CPU · pasa a GPU o a lotes nocturnos
Fuerza bruta	18 ms · más rápida que el índice	~180 ms · deja de ser viable
Índice HNSW	No aporta velocidad	Se vuelve imprescindible
Cuello de botella	Transporte HTTP (32 de 47 ms)	La búsqueda · el transporte deja de dominar
Latencia p50	59 ms	~70–90 ms si <code>ef_search</code> se mantiene

La conclusión operativa es que **la arquitectura elegida está dimensionada para el catálogo que viene, no para el de hoy**. A la escala actual el índice no aporta velocidad; a 150.000 productos es la única opción viable, y el trabajo de calibrarlo ya está hecho y medido. Lo que sí cambia de naturaleza es la codificación: doce minutos son un inconveniente, dos horas son una decisión de infraestructura.

LO QUE ESTE PROYECTO DEMUESTRA MÁS ALLÁ DE LAS CIFRAS

Tres de los cuatro hallazgos con más valor fueron incómodos: los embeddings empeoraron el sistema antes de mejorarlo, el motor respondía sin usar su índice con todos los indicadores en verde, y el índice que se pedía no aporta velocidad a esta escala. Ninguno se habría visto sin un oráculo exacto contra el que comparar y sin declarar el techo de cada métrica.

Un sistema que funciona sin saber por qué no es reproducible. Ese, y no el nDCG, es el resultado que se entrega.

Los tres artefactos de entrega, la configuración congelada (`config/final.yaml`), los ocho ADR y las 379 pruebas están en el repositorio. Toda cifra de este informe procede de un artefacto generado por `uv run aurum deliver` ; ninguna se escribió a mano.